



DATA STRUCTURES

LIBRARY MANAGEMENT SYSTEM

Student Id	Name-Surname	1st/2nd Ed.
230315038	Zeynep İclal BİLGİLİ	1st
230315074	Sude Naz AY	1st
230315020	Ayşe ARSLAN	1st

<https://github.com/iclal12/LibraryManagementSystem>

INTRODUCTION

This project implements a **console-based Library Management System** designed to efficiently manage books, members, and borrowing operations. The system provides librarians with a structured menu-driven interface to perform essential library tasks such as adding books, registering members, borrowing and returning books, searching for books, and undoing recent actions.

To ensure efficiency and reliability, the system utilizes multiple data structures including **HashMap, Stack, Queue, PriorityQueue, and Binary Search Tree (BST)**. Each data structure was carefully selected to address specific functional requirements such as fast lookup, undo functionality, waitlist handling, popularity analysis, and optimized searching.

The application is developed using **Java** and follows an object-oriented design approach. Clear separation of responsibilities among classes improves readability, scalability, and maintainability of the system.

SYSTEM REQUIREMENTS

This system fulfills all required functional specifications as listed below:

1. Catalog & Member Management

- Adding new books with unique IDs generated using hash codes
- Removing books from the system
- Registering new members
- Removing members if they have no borrowed books
- Book and member data stored using HashMap for fast Access

```
/* ===== CATALOG MANAGEMENT SYSTEM ===== */

public void addBook(String title, String author) { 1usage
    int id = Math.abs((title + GROUP_SEED).hashCode());
    Book book = new Book(id, title, author);
    books.put(id, book);
    bst.insert(book);
    System.out.println("Book added successfully. ID: " + id);
}

public void removeBook(int bookId) { 1usage
    Book b = books.get(bookId);

    if (b == null) {
        System.out.println("Book not found.");
        return;
    }

    books.remove(bookId);
    undoStack.push(new Action( type: "REMOVE_BOOK", b));
    System.out.println("Book removed successfully.");
}
```

```
/* ===== MEMBER MANAGEMENT SYSTEM ===== */

public void registerMember(String name) { 1usage
    int id = Math.abs((name + GROUP_SEED).hashCode());
    members.put(id, new Member(id, name));
    System.out.println("Member registered successfully. ID: " + id);
}

public void removeMember(int memberId) { 1usage
    Member m = members.get(memberId);

    if (m == null) {
        System.out.println("Member not found.");
        return;
    }

    if (!m.borrowedBooks.isEmpty()) {
        System.out.println("Member has borrowed books. Cannot remove.");
        return;
    }

    members.remove(memberId);
    undoStack.push(new Action( type: "REMOVE_MEMBER", m));
    System.out.println("Member removed successfully.");
}
```

2. Circulation (Borrowing & Returning)

- Availability control for each book
- Borrowing and returning operations
- Automatic waitlist handling using Queue
- Borrowing history tracking via borrow count

```
/* ===== CIRCULATION SYSTEM ===== */

public void borrowBook(int memberId, int bookId) { 1usage
    Member m = members.get(memberId);
    Book b = books.get(bookId);

    if (m == null || b == null) {
        System.out.println("Invalid member ID or book ID.");
        return;
    }

    if (b.available) {
        b.available = false;
        b.borrowCount++;
        m.borrowedBooks.add(bookId);
        popularBooks.add(b);
        undoStack.push(new Action( type: "BORROW", memberId, bookId));
        System.out.println("Book borrowed successfully.");
    } else {
        b.waitlist.add(memberId);
        System.out.println("Book not available. Added to waitlist.");
    }
}
```

3. Search & Discovery

- Book search functionality by title
- Binary Search Tree used for fast and efficient searching
- Case-insensitive title comparison

```
/* ===== SEARCH & DISCOVERY SYSTEM ===== */

public void searchBookByTitle(String title) { 1 usage
    Book b = bst.search(title);
    if (b == null)
        System.out.println("Book not found.");
    else
        System.out.println("Found: " + b.title + " by " + b.author +
            " | Available: " + b.available);
}
```

4. Analytics

- Tracking popular books based on borrow count
- PriorityQueue used to rank books by popularity

```
/* ===== ANALYTICS SYSTEM ===== */

public void showPopularBooks() { 1 usage
    if (popularBooks.isEmpty()) {
        System.out.println("No books borrowed yet.");
        return;
    }

    System.out.println("Most Popular Books:");
    for (Book b : popularBooks) {
        System.out.println(b.title + " (" + b.borrowCount + " borrows)");
    }
}
```

5. Undo Feature

- Undo last operations such as borrow, return, remove book, and remove member
- Stack data structure used to support LIFO undo behavior

```
/* ===== UNDO SYSTEM ===== */

public void undo() { 1 usage
    if (undoStack.isEmpty()) {
        System.out.println("Nothing to undo.");
        return;
    }

    Action action = undoStack.pop();

    switch (action.type) {

        case "BORROW":
            Book b = books.get(action.bookId);
            Member m = members.get(action.memberId);
            if (b != null && m != null) {
                b.available = true;
                b.borrowCount--;
                m.borrowedBooks.remove((Integer) action.bookId);
            }
            System.out.println("Undo borrow successful.");
            break;
    }
}
```

```
case "REMOVE_BOOK":
    books.put(action.bookId, action.removedBook);
    bst.insert(action.removedBook);
    System.out.println("Undo remove book successful.");
    break;

case "REMOVE_MEMBER":
    members.put(action.memberId, action.removedMember);
    System.out.println("Undo remove member successful.");
    break;
```

DATA STRUCTURES USED

```
/* ===== DATA STRUCTURES ===== */

HashMap<Integer, Book> books = new HashMap<>(); 8 usages
HashMap<Integer, Member> members = new HashMap<>(); 9 usages
Stack<Action> undoStack = new Stack<>(); 6 usages
BookBST bst = new BookBST(); 3 usages

PriorityQueue<Book> popularBooks = 3 usages
    new PriorityQueue<>((Book a, Book b) -> b.borrowCount - a.borrowCount);
```

HashMap

Purpose: Store and retrieve books and members efficiently

Reason for Use: Provides O(1) average-time complexity for search operations

Usage: Book and member storage

```
public void addBook(String title, String author) { 1 usage
    int id = Math.abs((title + GROUP_SEED).hashCode());
    Book book = new Book(id, title, author);
    books.put(id, book);
    bst.insert(book);
    System.out.println("Book added successfully. ID: " + id);
}
```

Stack (Undo System)

Purpose: Track reversible operations

Reason for Use: LIFO behavior is ideal for undo functionality

Usage: Undo borrow, return, remove operations

```
public void borrowBook(int memberId, int bookId) { 1 usage
    Member m = members.get(memberId);
    Book b = books.get(bookId);

    if (m == null || b == null) {
        System.out.println("Invalid member ID or book ID.");
        return;
    }

    if (b.available) {
        b.available = false;
        b.borrowCount++;
        m.borrowedBooks.add(bookId);
        popularBooks.add(b);
        undoStack.push(new Action("BORROW", memberId, bookId));
        System.out.println("Book borrowed successfully.");
    } else {
        b.waitList.add(memberId);
        System.out.println("Book not available. Added to waitlist.");
    }
}
```

Queue (Waitlist)

Purpose: Manage waiting members for unavailable books

Reason for Use: FIFO logic ensures fair allocation

Usage: Book waitlist handling

```
class Book { 18 usages
    int id; 2 usages
    String title; 5 usages
    String author; 2 usages
    boolean available = true; 6 usages
    int borrowCount = 0; 5 usages
    Queue<Integer> waitList = new LinkedList<>(); 3 usages

    public Book(int id, String title, String author) { 1 usage
        this.id = id;
        this.title = title;
        this.author = author;
    }
}
```

PriorityQueue

Purpose: Track most popular books

Reason for Use: Automatically sorts books based on borrow count

Usage: Analytics system

```
PriorityQueue<Book> popularBooks = 3 usages
    new PriorityQueue<>((Book a, Book b) -> b.borrowCount - a.borrowCount);
```

Binary Search Tree (BST)

Purpose: Enable fast searching by book title

Reason for Use: Efficient search performance ($O(\log n)$)

Usage: Search and discovery system

```
class BSTNode { 6 usages
    String title; 4 usages
    Book book; 2 usages
    BSTNode left, right; 3 usages

    @Contract(pure = true)
    public BSTNode( @NotNull Book book) { 1 usage
        this.book = book;
        this.title = book.title;
    }
}
```

SYSTEM DESIGN

The system follows an object-oriented design with clearly defined classes:

- **LibraryManagementSystem:** Main controller and menu handler
- **Book:** Represents book entities
- **Member:** Represents library members
- **Action:** Stores undoable actions
- **BookBST:** Handles book searching
- **BSTNode:** Node structure for BST

Library Management System

```
public class LibraryManagementSystem {

    /* ===== STUDENT ID DEFINITIONS ===== */

    static final int STUDENT_ID_1 = 230315038; 1 usage
    static final int STUDENT_ID_2 = 230315020; 1 usage
    static final int STUDENT_ID_3 = 230315074; 1 usage

    static final int GROUP_SEED = 2 usages
        (STUDENT_ID_1 + STUDENT_ID_2 + STUDENT_ID_3) % 1000000;

    /* ===== DATA STRUCTURES ===== */

    HashMap<Integer, Book> books = new HashMap<>(); 8 usages
    HashMap<Integer, Member> members = new HashMap<>(); 9 usages
    Stack<Action> undoStack = new Stack<>(); 6 usages
    BookBST bst = new BookBST(); 3 usages

    PriorityQueue<Book> popularBooks = 3 usages
        new PriorityQueue<>((Book a, Book b) -> b.borrowCount - a.borrowCount);
}
```

```
public void startMenu() { 1 usage
    Scanner sc = new Scanner(System.in);
    int choice;

    do {
        System.out.println("\n==== LIBRARY MANAGEMENT SYSTEM =====");
        System.out.println("1. Add Book");
        System.out.println("2. Register Member");
        System.out.println("3. Borrow Book");
        System.out.println("4. Return Book");
        System.out.println("5. Search Book by Title");
        System.out.println("6. Show Most Popular Books");
        System.out.println("7. Remove Book");
        System.out.println("8. Remove Member");
        System.out.println("9. Undo Last Action");
        System.out.println("0. Exit");
        System.out.print("Enter your choice: ");
    }
```

Class Entities (Book & Member)

Book Class – Represents Book Entities

The Book class models all information related to a library book. It stores the book's unique ID, title, author, availability status, and borrow count. Additionally, it maintains a `Queue<Integer> waitList` to manage members waiting for the book when it is unavailable. This design enables efficient circulation management and fair waitlist handling.

Member Class – Represents Library Members

The Member class represents registered library users. It stores the member's unique ID, name, and a list of currently borrowed books using a `LinkedList<Integer>`. This structure allows the system to easily track borrowing history and prevent invalid operations such as removing members with active loans.

```
/* ===== ENTITIES ===== */

class Book { 18 usages
    int id; 2 usages
    String title; 5 usages
    String author; 2 usages
    boolean available = true; 6 usages
    int borrowCount = 0; 5 usages
    Queue<Integer> waitList = new LinkedList<>(); 3 usages

    public Book(int id, String title, String author) { 1 usage
        this.id = id;
        this.title = title;
        this.author = author;
    }
}

class Member { 8 usages
    int id; 2 usages
    String name; 1 usage
    LinkedList<Integer> borrowedBooks = new LinkedList<>(); 6 usages

    public Member(int id, String name) { 1 usage
        this.id = id;
        this.name = name;
    }
}
```


Class Action

Action Class – Represents Undoable Operations

The `Action` class is used to record operations that can be reverted using the undo feature. It stores the type of action performed (such as `BORROW`, `RETURN`, `REMOVE_BOOK`, or `REMOVE_MEMBER`) along with relevant data including member ID, book ID, and references to removed objects when necessary. This design enables the Stack-based undo system to accurately restore previous system states.

```
class Action { 6 usages
    String type; 4 usages
    int memberId; 5 usages
    int bookId; 7 usages
    Book removedBook; 3 usages
    Member removedMember; 2 usages

    @Contract(pure = true)
    public Action(String type, int memberId, int bookId) { 2 usages
        this.type = type;
        this.memberId = memberId;
        this.bookId = bookId;
    }

    @Contract(pure = true)
    public Action(String type, @NotNull Book book) { 1 usage
        this.type = type;
        this.removedBook = book;
        this.bookId = book.id;
    }

    @Contract(pure = true)
    public Action(String type, @NotNull Member member) { 1 usage
        this.type = type;
        this.removedMember = member;
        this.memberId = member.id;
    }
}
```

Class BookBST & Class BSTNode

```
class BookBST { 2 usages
    BSTNode root; 3 usages

    public void insert(Book book) { 2 usages
        root = insertRec(root, book);
    }

    @Contract("null, _ -> new")
    private @NotNull BSTNode insertRec(BSTNode node, Book book) { 3 usages
        if (node == null) return new BSTNode(book);

        if (book.title.compareToIgnoreCase(node.title) < 0)
            node.left = insertRec(node.left, book);
        else
            node.right = insertRec(node.right, book);

        return node;
    }
}
```

```
class BSTNode { 6 usages
    String title; 4 usages
    Book book; 2 usages
    BSTNode left, right; 3 usages

    @Contract(pure = true)
    public BSTNode(@NotNull Book book) { 1 usage
        this.book = book;
        this.title = book.title;
    }
}
```

CORE FUNCTIONALITIES (5 IMPORTANT FUNCTIONS)

1. addBook()

Adds a new book to the system, generates a unique ID, stores it in `HashMap`, and inserts it into the `BST`.

```
public void addBook(String title, String author) { 1 usage
    int id = Math.abs((title + GROUP_SEED).hashCode());
    Book book = new Book(id, title, author);
    books.put(id, book);
    bst.insert(book);
    System.out.println("Book added successfully. ID: " + id);
}
```

2. borrowBook()

Allows a member to borrow a book if available. If not available, the member is added to the waitlist.

```
public void borrowBook(int memberId, int bookId) { 1usage
    Member m = members.get(memberId);
    Book b = books.get(bookId);

    if (m == null || b == null) {
        System.out.println("Invalid member ID or book ID.");
        return;
    }

    if (b.available) {
        b.available = false;
        b.borrowCount++;
        m.borrowedBooks.add(bookId);
        popularBooks.add(b);
        undoStack.push(new Action( type: "BORROW", memberId, bookId));
        System.out.println("Book borrowed successfully.");
    } else {
        b.waitlist.add(memberId);
        System.out.println("Book not available. Added to waitlist.");
    }
}
```

3. returnBook()

Handles book return operations and assigns the book to the next member in the waitlist if applicable.

```
public void returnBook(int memberId, int bookId) { 1usage
    Member m = members.get(memberId);
    Book b = books.get(bookId);

    if (m == null || b == null) {
        System.out.println("Invalid member ID or book ID.");
        return;
    }

    m.borrowedBooks.remove((Integer) bookId);

    if (!b.waitlist.isEmpty()) {
        int nextMember = b.waitlist.poll();
        members.get(nextMember).borrowedBooks.add(bookId);
        System.out.println("Book assigned to next member in waitlist.");
    } else {
        b.available = true;
        System.out.println("Book returned successfully.");
    }

    undoStack.push(new Action( type: "RETURN", memberId, bookId));
}
```

4. searchBookByTitle()

```
public void searchBookByTitle(String title) { 1usage
    Book b = bst.search(title);
    if (b == null)
        System.out.println("Book not found.");
    else
        System.out.println("Found: " + b.title + " by " + b.author +
            " | Available: " + b.available);
}
```

5. showMostPopularBooks()

```
public void showPopularBooks() { 1usage
    if (popularBooks.isEmpty()) {
        System.out.println("No books borrowed yet.");
        return;
    }

    System.out.println("Most Popular Books:");
    for (Book b : popularBooks) {
        System.out.println(b.title + " (" + b.borrowCount + " borrows)");
    }
}
```

OUTPUT SAMPLE

1.Search Book By Title Output

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Register Member
3. Borrow Book
4. Return Book
5. Search Book by Title
6. Show Most Popular Books
7. Remove Book
8. Remove Member
9. Undo Last Action
0. Exit
Enter your choice: 5
Enter book title: C and Data Structure
Found: C and Data Structure by P Padmanabham | Available: true
```

2.Borrow Book Output

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Register Member
3. Borrow Book
4. Return Book
5. Search Book by Title
6. Show Most Popular Books
7. Remove Book
8. Remove Member
9. Undo Last Action
0. Exit
Enter your choice: 3
Enter member ID: 901591015
Enter book ID: 1482281039
Book borrowed successfully.
```

3.Return Book Output

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Register Member
3. Borrow Book
4. Return Book
5. Search Book by Title
6. Show Most Popular Books
7. Remove Book
8. Remove Member
9. Undo Last Action
0. Exit
Enter your choice: 4
Enter member ID: 901591015
Enter book ID: 1482281039
Book returned successfully.
```

4.Show Most Popular Books Output

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Register Member
3. Borrow Book
4. Return Book
5. Search Book by Title
6. Show Most Popular Books
7. Remove Book
8. Remove Member
9. Undo Last Action
0. Exit
Enter your choice: 6
Most Popular Books:
Introduction to Data Structures and Algorithms with Java (1 borrows)
```

5.Remove Book Output

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Register Member
3. Borrow Book
4. Return Book
5. Search Book by Title
6. Show Most Popular Books
7. Remove Book
8. Remove Member
9. Undo Last Action
0. Exit
Enter your choice: 7
Enter book ID: 1482281039
Book removed successfully.
```

6.Remove Member Output

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Register Member
3. Borrow Book
4. Return Book
5. Search Book by Title
6. Show Most Popular Books
7. Remove Book
8. Remove Member
9. Undo Last Action
0. Exit
Enter your choice: 8
Enter member ID: 901591015
Member removed successfully.
```


7.Undo Function Output

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Register Member
3. Borrow Book
4. Return Book
5. Search Book by Title
6. Show Most Popular Books
7. Remove Book
8. Remove Member
9. Undo Last Action
0. Exit
Enter your choice: 9
Undo remove member successful.
```

8.Exit

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. Register Member
3. Borrow Book
4. Return Book
5. Search Book by Title
6. Show Most Popular Books
7. Remove Book
8. Remove Member
9. Undo Last Action
0. Exit
Enter your choice: 0
Exiting system...
```

UNIQUE ID USAGE

```
/* ===== STUDENT ID DEFINITIONS ===== */

static final int STUDENT_ID_1 = 230315038; 1 usage
static final int STUDENT_ID_2 = 230315020; 1 usage
static final int STUDENT_ID_3 = 230315074; 1 usage

static final int GROUP_SEED = 2 usages
    (STUDENT_ID_1 + STUDENT_ID_2 + STUDENT_ID_3) % 1000000;
```

Unique ID Evidence Student IDs are combined into a GROUP_SEED value to generate unique IDs.

ANALYSIS & PERFORMANCE EVALUATION

Time & Space Complexity Analysis

Operation / Data Structure	Data Structure Used	Time Complexity	Space Complexity	Explanation
Add Book	HashMap + BST	$O(1) + O(\log n)$	$O(n)$	HashMap enables constant-time insertion, while BST maintains sorted structure
Remove Book	HashMap	$O(1)$	$O(n)$	Direct access using book ID
Register Member	HashMap	$O(1)$	$O(n)$	Member stored with unique hash-based ID
Borrow Book	HashMap + Queue	$O(1)$	$O(n)$	Availability check is constant-time; Queue handles waitlist
Return Book	HashMap + Queue	$O(1)$	$O(n)$	FIFO queue assigns book to next member
Search Book by Title	Binary Search Tree	$O(\log n)$	$O(n)$	BST enables efficient title-based searching
Undo Operation	Stack	$O(1)$	$O(n)$	LIFO structure ensures fast undo
Popular Books Analysis	PriorityQueue	$O(\log n)$	$O(n)$	Maintains books sorted by borrow count

Performance Discussion

The system demonstrates efficient performance by leveraging appropriate data structures for each operation. HashMap ensures fast access, BST improves search efficiency, PriorityQueue enables quick popularity analysis, and Stack guarantees reliable undo functionality. These design decisions reflect a conscious and optimized approach to data structure selection.

Usage Guide

Compilation

The application is compiled using the Java compiler with the following command:

```
javac LibraryManagementSystem.java
```

Execution

After successful compilation, the program is executed using the command below:

```
java LibraryManagementSystem
```

Once the program starts, a menu-driven console interface is displayed, allowing the user to interact with the system.

System Usage

The user interacts with the system through a numbered menu:

1. Add Book – Enter book title and author to add a new book.
2. Register Member – Register a new library member.
3. Borrow Book – Borrow a book using member ID and book ID.
4. Return Book – Return a borrowed book.
5. Search Book by Title – Search for a book using its title.
6. Show Most Popular Books – Display books sorted by borrow count.
7. Remove Book – Remove a book from the system.
8. Remove Member – Remove a member if no borrowed books exist.
9. Undo Last Action – Revert the most recent operation.
10. Exit – Safely terminate the program.

Tested Environment

The application was tested under the following conditions:

- Operating System: Windows 10
- Java Version: Java SE 8 or higher
- Interface: Command Line / Console

The program executed correctly in this environment, and all system features were tested via the console interface.

CONCLUSION

In conclusion, the Library Management System successfully fulfills all project requirements. The use of appropriate data structures improved system performance, reliability, and code organization. Features such as undo functionality, waitlist handling, and analytics significantly enhance usability. The project demonstrates effective application of data structures in a real-world scenario.